CTO REVIEW

Marveos CTO Architecture and Risk Report

Date: 2026-05-26
Scope: marveos, marveo-connector, marveo-templates
Assessment basis: current implementation only

> **Executive position:** Marveos is already a meaningful operational platform, but it is not yet a fully automated production deployment system.
>
> It is strong enough for controlled operator-led onboarding and support-assisted client rollout. It is not yet strong enough for broad self-serve deployment promises because the deployment workflow is more mature than the deployment executor behind it.

Decision Summary

AREA	STATUS	CTO ASSESSMENT	PRIMARY OWNER
Persistence model	Yellow	Works today, but centralizes too much risk in one JSONB control document.	Platform Engineering
Authentication and authorization	Yellow	Real role-aware security model, but not yet distributed-safe.	Platform Engineering + Security
Commercial onboarding	Yellow	Good enough for managed funnel operations.	Product Engineering
Workspace orchestration	Yellow	Real orchestration state machine, not real infrastructure truth.	Platform Engineering
WordPress connector	Yellow	Valuable adapter layer, but not a mature sync substrate.	Integrations Engineering
Template operations	Yellow	Operationally useful catalog, but deployment coupling is incomplete.	Product Engineering
Deployment automation	Red	Major gap. Workflow exists; infrastructure executor does not.	Platform Engineering + DevOps
Security hardening	Yellow/Red	Intent is strong; implementation is still partly single-instance.	Security + Platform Engineering
Client rollout readiness	Yellow	Suitable for operator-led clients, not broad self-serve.	CTO / Delivery

Core Architecture Reality

System control plane

The true control plane is the Postgres-backed admin store in `lib/adminStore.ts`. Almost all platform state is persisted into one JSONB document keyed by a single row and read back into an application-owned aggregate object.

Code anchors: `lib/adminStore.ts`, including the Postgres table definition, JSONB upsert path, store read/update API, and the default control-document shape.

This architecture is fast to evolve, but it places data integrity, concurrency discipline, and tenant isolation pressure on application logic instead of database structure.

Authentication model

The platform uses native session resolution and a WordPress bridge path, then normalizes identity into Marveo roles enforced by server-side guards.

Code anchors: `lib/auth.ts`, `lib/nativeAuth.ts`, `app/api/auth/login/route.ts`, `lib/master/permissions/guards.ts`.

The auth model is real. The main gap is operational hardening: OTP, rate-limit, and lockout state are process-local rather than shared.

Workspace model

Workspaces are created as orchestration objects containing ownership scope, onboarding steps, support state, rollout metadata, and deployment-readiness flags.

Code anchors: `lib/cloudOrchestration.ts`, `lib/workspaceEntitlements.ts`, `app/api/cloud/workspaces/route.ts`.

The system has a real workspace domain, but a workspace still represents operational state much more than verified infrastructure reality.

Subsystem Findings

Persistence and data integrity

The JSONB store makes the system flexible, but domain boundaries are soft. Commercial, auth, workspace, finance, and operational data all move through one aggregate persistence model.

CTO assessment: acceptable for the current phase if the platform is deliberately operated as a controlled service, not as a high-scale autonomous multi-tenant product.

Owner: Platform Engineering

Authentication and platform access

The auth design is better than a typical early-stage internal tool. Native and bridged identities already coexist, internal versus client roles are explicit, and module-action authorization is enforced server-side.

Main risk: distributed inconsistency in login protection and challenge handling.

Owner: Security and Platform Engineering

Commercial onboarding and entitlement

Commercial onboarding is real and is one of the stronger slices of the platform. Public plans, templates, onboarding sessions, subscription states, and entitlement resolution are consistently wired into user-facing onboarding behavior.

> **Code anchors:** `lib/commercialOnboarding.ts`, `app/api/subscription/current/route.ts`.

Main risk: billing truth remains application-owned rather than independently anchored.

Owner: Product Engineering and Platform Engineering

Onboarding workflow

The onboarding page in `app/setup/mvp/page.tsx` is a real orchestration client. It restores draft state, validates entitlement, loads templates, verifies connectors, creates workspaces, assigns support, and fetches readiness state.

Main risk: workflow-critical sequencing is concentrated in a large client component, and the current step guard means the visible flow is narrower than the component's latent model.

Owner: Product Engineering

WordPress connector

The connector is already useful for runtime content reads, status verification, content mapping, module activation, and deployment-status reporting.

> **Code anchors:** `marveo-connector/includes/class-rest-api.php`, `marveo-connector/includes/class-rest-api-extended.php`, `marveo-connector/includes/class-hooks.php`.

Main risk: runtime reads are comparatively open once connector status is active enough, and long-term sync hooks are still placeholders.

Owner: Integrations Engineering

Template system

The template system is operationally real: metadata, import, resync, publish rules, and public filtering are implemented. It is useful for curated onboarding.

Main risk: template availability can be mistaken for deployment readiness, but the deployment engine does not yet complete the promise.

Owner: Product Engineering

Deployment and launch

This is the highest-risk gap in the platform. Deployment links, launch checklist, and launch guard exist, but the execution layer behind them is incomplete.

> **Critical code anchors:** `app/api/cloud/deployment-links/route.ts`, `app/api/cloud/deployment-links/[linkId]/route.ts`, `app/api/cloud/workspaces/[workspaceId]/launch-guard/route.ts`, `lib/cloudOrchestration.ts`.
>
> **Critical implementation reality:** the finalize route explicitly calls out future work for infrastructure provisioning, DNS configuration, SSL certificate generation, webhook registration, and content sync triggering.

CTO conclusion: this is the main blocker to self-serve rollout.

Owner: Platform Engineering and DevOps

Infrastructure and security posture

Security intent is good, but operational hardening is not complete. Public route throttling, audit logs, and permission guards exist, but several core control mechanisms remain in memory.

Main risk: multi-instance production would weaken protection guarantees unless the state model changes.

Owner: Security and Platform Engineering

Client Readiness Assessment

The platform can support a real client today if Marveo staff remain in the loop. It can capture entitlement, create workspaces, verify existing WordPress connectors, assign support, and generate readiness state.

It should not yet be described as a fully self-serve production deployment engine for new client environments.

> **Recommended external posture:** Marveos is a managed onboarding and operational orchestration platform with partial automation, not yet a fully autonomous deployment platform.

Top Risks and Risk Owners

PRIORITY	RISK	WHY IT MATTERS	OWNER
P1	Deployment execution gap	Core self-serve value proposition is not actually automated yet.	Platform Engineering + DevOps
P1	Process-local security controls	Multi-instance rollout would weaken lockout, OTP, and rate-limit guarantees.	Security + Platform Engineering
P1	Overloaded JSONB control plane	Scale, reporting, and concurrent mutation complexity will grow quickly.	Platform Engineering
P2	Connector runtime authorization looseness	Runtime reads are easier to expose than intended if connector status is active enough.	Integrations Engineering + Security
P2	Client workflow logic concentrated in frontend	Increases regression risk and makes future onboarding variants harder to maintain.	Product Engineering
P2	Template readiness not equal to deployment readiness	Commercial and delivery expectations can diverge.	Product Engineering + Delivery
P3	Mixed identity persistence complexity	Native and bridged auth coexistence is useful, but long-term ownership needs cleanup.	Platform Engineering

90-Day CTO Action Plan

Phase 1: harden what already exists

1. Move login OTP, lockout, and public rate-limit state into shared storage.
2. Tighten connector runtime authorization for non-public endpoints.
3. Add explicit operational telemetry around workspace creation, connector verification, support assignment, and launch approval.

Owner: Security + Platform Engineering

Phase 2: make deployment real

1. Build a deployment executor behind the existing deployment-link and launch surfaces.
2. Represent provisioning steps as real jobs with retries, status, and audit events.
3. Wire DNS, certificate, and environment/bootstrap tasks into the executor.

Owner: Platform Engineering + DevOps

Phase 3: reduce concentration risk

1. Pull workflow-critical sequencing out of the large onboarding client component.
2. Introduce clearer backend workflow handlers for onboarding progression.
3. Begin separating high-churn domains from the monolithic admin store aggregate.

Owner: Product Engineering + Platform Engineering

> **Final recommendation:** continue forward as a managed-service-first platform.
>
> **Final CTO verdict:** approved for controlled managed onboarding. Not approved yet for broad self-serve production rollout.

Reference Files Reviewed

- `lib/adminStore.ts`
- `lib/auth.ts`
- `lib/nativeAuth.ts`
- `app/api/auth/login/route.ts`
- `lib/workspaceEntitlements.ts`
- `lib/cloudOrchestration.ts`
- `app/api/cloud/workspaces/route.ts`
- `app/api/cloud/workspaces/[workspaceId]/launch-checklist/route.ts`
- `app/api/cloud/workspaces/[workspaceId]/launch-guard/route.ts`
- `app/api/cloud/deployment-links/route.ts`

- `app/api/cloud/deployment-links/[linkId]/route.ts`
- `lib/commercialOnboarding.ts`
- `app/api/subscription/current/route.ts`
- `app/setup/mvp/page.tsx`
- `marveo-connector/includes/class-rest-api.php`
- `marveo-connector/includes/class-rest-api-extended.php`
- `marveo-connector/includes/class-hooks.php`

Prepared by GitHub Copilot using GPT-5.4. Product: Marveos.